

21-ma'ruza. Qiymat turlari va havola turlar

Reja:

1. Qiymat turlari. Qiymatlarni nusxalash
2. ref va out kalit so'zlari
3. ref kalit so'zidan foydalanish
4. out kalit so'zidan foydalanish

Oldingi mavzularda quyidagi elementar berilganlar turlarini ko'rib chiqdik: **int**, **bayt**, **double**, **string**, **object** va boshqalar. Bundan tashqari, murakkab turlar mavjud: strukturalar (struct), sanab o'tiluvchi turlar (enum), sinflar (class). Ushbu barcha ma'lumot turlari qiymat turlari deb ham ataladigan qiymat turlari (value types) va havola turlari (reference types) ga bo'linishi mumkin. Ularning orasidagi farqni tushunish muhimdir.

Qiymat turlari:

Butun sonlar turlari: (**bayt**, **sbayt**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**)

Suzuvchi nuqta turlari: (**float**, **double**, **decimal**)

Tur: **bool**

Tur: **char**

Sanab o'tiluvchi tur: **enum**

Strukturalar: **struct**

Havola turlari:

Tur: **object**

Tur: **string**

Sinflar: **class**

Interfeyslar: **interface**

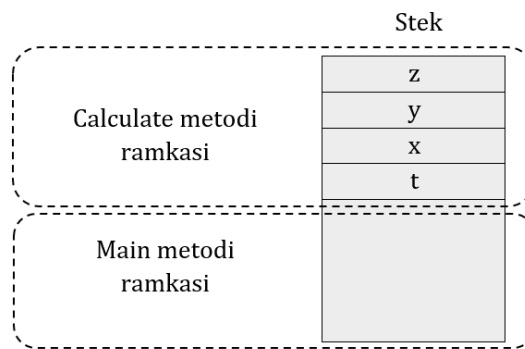
Delegatlar: **delegate**

Ularning orasidagi farqni ajratish uchun .NET da xotirani tashkil qilishni tushunish kerak. Bu yerda, xotira ikki turga bo'linadi: stek va to'p (mavjud xotiraning qolgan qismi). Qiymat turlarini ifodalovchi metod parametrlari va o'zgaruvchilari o'z qiymatini stekga o'tkazadi. Stek - bu pastdan yuqoriga o'sadigan berilganlar strukturasi: qo'shilgan har bir yangi element avvalgisining ustiga joylashtiriladi.

Dastur birinchi marta bajarish uchun ishga tushirilganda, stek ko'rsatkichi stek uchun ajratilgan xotira blokining oxiriga o'rnatiladi. Berilganlar stekga o'tkazilganda, ko'rsatkich qayta o'rnatiladi, shunda u yana yangi bo'sh joyga ishora qiladi. Har bir alohida metod chaqirilganda, stekda uning parametrlari va o'zgaruvchilari qiymatlari saqlanadigan xotira maydoni yoki stek ramkasi ajratiladi.

Misol uchun:

```
class Program
{
    static void Main(string[] args)
    {
        Calculate(5);
    }
    static void Calculate(int t)
    {
        int x = 6;
        int y = 7;
        int z = y + t;
    }
}
```



Ushbu Calculate() metodini chaqirish t, x, y va z qiymatlarini stekga jo'natadi. Ular berilgan metod tanasida aniqlanadi. Metod ishini tugatgach, stek uchun ajratilgan xotira maydoni keyinchalik boshqa metodlar bilan ishlatilishi mumkin.

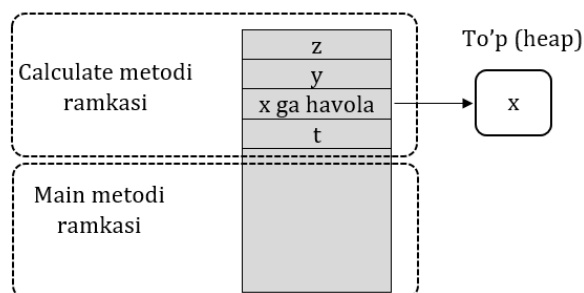
Bundan tashqari, agar metod parametri yoki o'zgaruvchisi qiymat turini ifodalasa, ushbu parametr yoki o'zgaruvchining bevosita qiymati stekda saqlanadi. Masalan, Calculate() metodining o'zgaruvchilari va parametrlari qiymat turi - int turini ifodalaydi, shuning uchun ularning sonli qiymatlari stekda saqlanadi.

Havola turlar bir xil bo'lmagan obyektlarning tartibsiz to'plami sifatida ko'rib chiqilishi mumkin bo'lgan to'pda saqlanadi. Havola turdagi obyekt yaratilganda, stekga to'pdagi manzilga havola joylashtiriladi. Agar mos havola turidagi obyekt endi ishlatilmasa, avtomatik xotira tozalash ishga tushadi: u to'pda obyektga boshqa havolalar yo'qligini ko'radi, bu obyektning shartli ravishda o'chiradi va xotirani tozalaydi, ya'ni ushbu xotira segmenti boshqa ma'lumotlarni saqlash uchun ishlatilishi mumkin.

Shunday qilib, agar Calculate metodini quyidagicha o'zgartirilsa:

```
static void Calculate(int t)
{
    object x = 6;
    int y = 7;
    int z = y + t;
}
```

Endi x o'zgaruvchisining qiymati to'pda saqlanadi, chunki u obyektga havola turini ifodalaydi va stek obyektga havolani to'pda saqlaydi.



Murakkab turlar

Endi qiymat turi va mos havola turi murakkab turlarni - struktura va sinfni ifodalovchi holatni ko'rib chiqamiz:

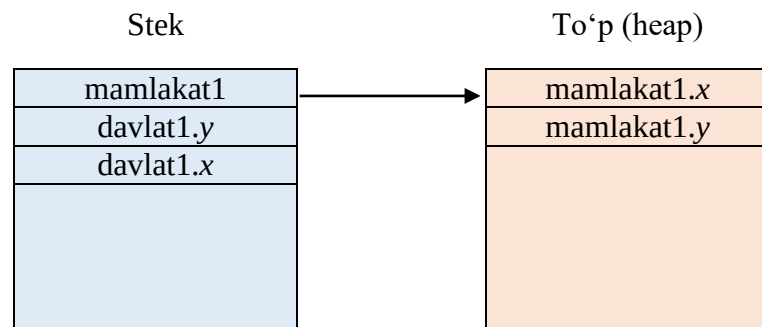
```
// Davlat strukturasi, uning ma'lumotlari stekga joylashtiriladi
Davlat davlat1 = new Davlat();
// Mamlakat sinfi, havola stekga joylashtiriladi va mamlakat1 obyektining barcha
ma'lumotlari to'pda joylashadi
Mamlakat mamlakat1 = new Mamlakat();
struct Davlat
{
    public int x;
```

```

    public int y;
}
class Mamlakat
{
    public int x;
    public int y;
}

```

Bu yerda, Main metodida *davlat1* obyekt uchun xotira stekga ajratiladi. Keyinchalik, stekda *mamlakat1* obyekt uchun mos havola yaratiladi va *new* kalit soʻzi bilan konstruktor chaqiruvidan foydalanib, toʻpda joy ajratiladi (*new Mamlakat()*). *mamlakat1* obyekt uchun stek havolasi obyekt joylashgan toʻpdagi joylashuv manzilini ifodalaydi.



Shunday qilib, *mamlakat1* obyektining barcha maydonlari va toʻpdagi *mamlakat1* obyektiga havola stekda boʻladi.

Qiymatlarni nusxalash

Qiymatlarni nusxalashda havola turini hisobga olish kerak. Qiymat turidagi obyektga havola tayinlaganda, u berilganlarning nusxasini oladi. Berilganlarni mos havola turidagi obyektga tayinlashda u obyektning nusxasini emas, balki toʻpdagi ushbu obyektga havolani oladi. Misol uchun:

```

using System;
namespace Havolalar
{
    class Program
    {
        public static void Main(string[] args)
        {
            // Davlat strukturasi obyektlari
            Davlat davlat1 = new Davlat();
            Davlat davlat2 = new Davlat();
            davlat2.x = 1;
            davlat2.y = 2;
            davlat1 = davlat2;
            davlat2.x = 5;
            Console.WriteLine(davlat1.x); // 1
            Console.WriteLine(davlat2.x); // 5
            // Mamlakat sinfi obyektlari
            Mamlakat mamlakat1 = new Mamlakat();
            Mamlakat mamlakat2 = new Mamlakat();
            mamlakat2.x = 1;
            mamlakat2.y = 4;
            mamlakat1 = mamlakat2;
            mamlakat2.x = 7;
        }
    }
}

```

```

        Console.WriteLine(mamlakat1.x); // 7
        Console.WriteLine(mamlakat2.x); // 7
    }
}
struct Davlat
{
    public int x;
    public int y;
}
class Mamlakat
{
    public int x;
    public int y;
}
}

```

davlat1 struktura bo'lgani uchun *davlat1* = *holat2* tayinlanganda, *davlat2* strukturasining nusxasini oladi va *mamlakat1* = *mamlakat2* ni belgilashda *mamlakat1* sinfining obyekt; *mamlakat2* tomonidan ko'rsatilgan bir xil obyektga havola oladi. Shuning uchun *mamlakat2* o'zgarishi bilan *mamlakat1* ham o'zgaradi.

Qiymat turlari ichidagi mos havola turlar

Strukturaning ichida mos havola turidagi o'zgaruvchiga ega bo'lish mumkin, masalan:

```

Davlat davlat1 = new Davlat();
Davlat davlat2 = new Davlat();
davlat2.mamlakat = new Mamlakat();
davlat2.mamlakat.x = 5;
davlat1 = davlat2;
davlat2.mamlakat.x = 8; // haliyam davlat1.mamlakat.x = 8, chunki davlat1.mamlakat va
davlat2.mamlakat to'pda bir narsaga ishora qiladi

```

```

Console.WriteLine(davlat1.mamlakat.x); // 8
Console.WriteLine(davlat2.mamlakat.x); // 8

```

```

struct Davlat
{
    public int x;
    public int y;
}
class Mamlakat
{
    public int x;
    public int y;
}

```

Bu yerda, *Main()* metodida *davlat1* obyekt uchun xotira stekda ajratiladi. Keyinchalik, *mamlakat1* obyekt (*Mamlakat mamlakat1*) uchun stekda havola yaratiladi va konstruktorni *new* kalit so'zi bilan chaqirish orqali to'pda bo'sh joy ajratiladi (*new Mamlakat ()*). *mamlakat1* obyekt uchun stek havolasi obyekt joylashtirilgan to'pdagi joylashuv manzilini ifodalaydi.

ref va out kalit so'zlari

int yoki char kabi oddiy turdagi berilganlar qiymatga qarab metodga o'tkaziladi. Bu qiymatni oladigan parametrga kiritilgan o'zgartirish uni chaqirish uchun ishlatilgan argumentga ta'sir qilmaydi degan ma'noni anglatadi. Ammo bu jarayonlar oddiy turlarning qiymatlarini havolalarga o'tish uchun **ref** va **out** kalit so'zlari yordamida o'zgartirilishi mumkin. Bu metodni chaqirganda belgilangan argumentni o'zgartirishga imkon beradi.

ref va **out** kalit so'zlardan foydalanishning o'ziga xos xususiyatlariga o'tishdan oldin, ba'zan havolalar orqali oddiy turdagi qiymatni o'tkazish talab qilinadigan sabablarni tushunish

kerak. Buning ikkita sababi bor: metodga o'z argumentlarining qiymatini o'zgartirishga yoki bir nechta qiymatlarni qaytarishga ruxsat berish.

Ko'pincha metod unga berilgan qiymatlar asosida ishlashi talab qilinadi. Oddiy misol almashtirish() metodi, uning argumentlari qiymatlarini o'zgartiradi. Oddiy turlarning argumentlari qiymat bo'yicha uzatilayotganligi sababli, C# da tanlangan standart chaqiruv mexanizmidan foydalanib, parametrga argument berish, uning ikkita argument qiymatlarini o'zaro almashtiradigan metodni yozish mumkin emas, masalan, int turi. Ushbu qiyinchilik **ref** modifikatori tomonidan hal qilinadi.

Qiymat dasturni chaqiruvchi qismi metodidan qaytarish kodi yordamida qaytariladi. Ammo metod bir vaqtning o'zida faqat bitta qiymatni qaytarishi mumkin. Ammo metoddan, sonli qiymatning butun va kasr qismlarini suzuvchi nuqta bilan qaytarish kerak bo'lsa, nima qilish kerak? Bunday metod **out** modifikator yordamida yozilishi mumkin.

ref modifikatoridan foydalanish

O'zgartirish parametrini o'zgartirish chaqiruvining qiymati bo'yicha emas, balki havola bo'yicha majburlaydi. Ushbu modifikator metod e'lonida ham, metodni ishlatganda ham yoziladi. Quyidagi dasturda Sqr() metodi yaratilib, uning argumenti o'rniga uning butun sonining kvadratini qaytaradi.

ref modifikatorni qo'llash va joylashishiga e'tibor berish lozim.

```
using System;
class RefTest {
    // Ushbu metod o'z argumentini o'zgartiradi.
    // ref modifikatordan foydalanish
    public void Sqr(ref int i)
    {
        i = i * i;
    }
}
class RefDemo {
    static void Main() {
        RefTest ob = new RefTest ();
        int a = 10;
        Console.WriteLine("a ning oldingi qiymati: " + a);
        ob.Sqr(ref a); // ref dan foydalanishga e'tibor bering
        Console.WriteLine("a ning keyingi qiymati: " + a);
        Console.ReadKey();
    }
}
```

Ko'rinib turibdiki, modifikator metoddagi parametr e'lonidan oldin va metod chaqirilganda argumentdan oldin ko'rsatiladi. Quyida ushbu dasturni bajarish natijasi keltirilgan, bu argumentning qiymati Sqr() metodi yordamida o'zgarganligini ko'rish mumkin.

a ning oldingi qiymati: 10

a ning keyingi qiymati: 100

Endi, *ref* operatoridan foydalanib, oddiy turdagi ikkita argumentlarning qiymatlarini o'zaro almashtiradigan metod yozish mumkin. Misol tariqasida, quyida Swap() metodi chaqirilganda uning ikkita butun sonli argumentlari qiymatlarini o'zgartiradigan dastur mavjud.

```
using System;
class ValueSwap
{
    // Ushbu metod o'z argumentlarini almashtiradi
    public void Swap1(ref int a, ref int b)
    {
        int t;
```

```

        t = a;
        a = b;
        b = t;
    }
    // Ushbu metod o'z argumentlarini almashtirmaydi
    public void Swap2(int a, int b)
    {
        int t;
        t = a;
        a = b;
        b = t;
    }
}
class ValueSwapDemo
{
    static void Main()
    {
        ValueSwap ob = new ValueSwap();
        int x = 10, y = 20;
        ob.Swap2(x, y);
        Console.WriteLine("Swap2 funksiyada x va y qiymati: " + x + " " + y);
        ob.Swap1(ref x, ref y);
        Console.WriteLine("Swap1 funksiyada x va y qiymati: " + x + " " + y);
        Console.ReadKey();
    }
}

```

Mana, ushbu dastur bajarilishining natijasi.

Swap2 funksiyada x va y qiymati: 10 20

Swap1 funksiyada x va y qiymati: 20 10

ref modifikatoriga nisbatan quyidagilarni yodda tutish kerak. Ushbu modifikator yordamida mos havola orqali olingan argumentga metod chaqirilishidan oldin qiymat belgilanishi kerak. Bunday argumentni parametr sifatida qabul qiladigan metodda parametr haqiqiy qiymatga tegishli deb taxmin qilinadi. Shuning uchun metodda *ref* o'zgartirgichdan foydalanganda argumentning boshlang'ich qiymatini aniqlay olmaysiz.

Xulosa: *ref* kalit so'zi qachonki, biz metod parametri faqat kerakli natija uchun vosita sifatida emas, balki parametrning qiymatini o'zgartirish ustuvor vazifa bo'lsa zarur bo'ladi.

out modifikatoridan foydalanish

Ba'zida metoddan qiymat olish uchun unga o'tish uchun emas, balki mos havola parametridan foydalanish kerak bo'ladi. Aytaylik, ba'zi funksiyalarni bajaradigan metod mavjud, masalan, tarmoq rozetkasini ochish va ushbu operatsiyani muvaffaqiyatli yoki muvaffaqiyatsiz yakunlash uchun kodni mos havola parametri sifatida qaytarish. Bunday holda, metodga hech qanday ma'lumot uzatilmaydi, lekin ma'lum ma'lumotlarni qaytarishi kerak. Ushbu holatda asosiy qiyinchilik, metodni chaqirishdan oldin, ushbu turdagi *ref* parametrini ma'lum bir qiymat bilan boshlash kerak. Shuning uchun ushbu turdagi parametrni ishlatishda argument uchun *ref* qiymatini belgilash va shu bilan ushbu cheklovni yengib o'tish kerak. C# da ushbu qiyinchilikdan chiqishning yanada qulay usuli bor ya'ni *out* parametrlarni o'zgartirish vositasidan foydalanish.

Parametrni o'zgartiradigan modifikator, faqat bitta istisnosiz, *ref* modifikatoriga o'xshaydi: u faqat metoddan tashqaridagi qiymatni uzatish uchun xizmat qiladi. Shuning uchun kiruvchi parametr sifatida ishlatiladigan o'zgaruvchiga qiymat tayinlash kerak emas (va foydasiz). Bundan tashqari, ushbu metodda *out* parametr, ya'ni initsializatsiya qilinmagan deb hisoblanadi. uning boshlang'ich qiymati yo'q deb taxmin qilinadi. Bu metod bajarilishidan oldin qiymat ushbu

parametrga tayinlanishi kerakligini anglatadi. Shuning uchun funksiyani chaqirgandan so'ng, *out* parametr ba'zi qiymatlarni o'z ichiga oladi.

Quyida *out* parametridan foydalanishga misol keltirilgan. Ushbu misol dasturda suzuvchi nuqtali sonni butun va kasr qismlarga ajratish uchun Ajratish sinfidagi GetParts() metodi qo'llaniladi. Berilgan sonning har bir qismi qanday qilib qaytarilishiga e'tibor bering.

```
using System;
class Ajratish
{
    /* Suzuvchi nuqtali sonni butun va kasr qismlarga ajratish */
    public int GetParts(double n, out double frac)
    {
        int w;
        w = (int)n;
        frac = n - w; // frac parametri orqali sonning kasr qismini olish
        return w; // sonning butun qismini qaytarish
    }
}
class UseOut
{
    static void Main()
    {
        Ajratish ob = new Ajratish();
        int i;
        double f;
        i = ob.GetParts(10.125, out f);
        Console.WriteLine("Sonning butun qismi " + i);
        Console.WriteLine("Sonning kasr qismi " + f);
        Console.ReadKey();
    }
}
```

Dastur natijasi:

Sonning butun qismi 10

Sonning kasr qismi 0,125

Ko'rib turganingizdek *out* parametri qiymatga ega emas, qiymatga ega bo'lgan o'zgaruvchilarni ham *out* bilan chaqirishimiz mumkin, *out* parametrli o'zgaruvchi metod tashqarisida e'lon qilinmagan, lekin tashqarida foydalanish mumkin, *out* parametr sifatida oldin metod tashqarisida e'lon qilingan o'zgaruvchilardan foydalanish mumkin.

Eslatma: Metodda *out* parametrli o'zgaruvchi e'lon qilib foydalanmaslik xatolikka olib keladi.

Xulosa: Metodning *out* parametri orqali e'lon qilingan o'zgaruvchini metod ichida o'zgartirish mumkin va metod tashqarisida ham bu o'zgarish saqlanib qoladi. Metodda odatiy e'lon qilingan o'zgaruvchilarda esa bu hodisa ro'y bermaydi.

Shuningdek, bu imkoniyatlar **ref** kalit so'zida ham mavjud, lekin unda o'zgaruvchining dastlabki qiymati aniq berilishi shart.